

CSCI 112 Assignment – Arrays and Loops

Points: 10

Automated tests will run on GitHub Actions after every push. Check the Actions tab to see which tests pass and which fail.

Instructions

1. **Fork** the repository on GitHub - <https://github.com/jpach-cs/112-ass05-template>.
2. Clone your forked repository to your local machine.
3. Open the file `code.h` and add:
 1. Include guards at the top: `#ifndef CODE_H #define CODE_H`
 2. All function prototypes
 3. Include guards at the bottom: `#endif`
4. Open the file `code.c`.
 1. Implement every function listed below:
 - Read the comments above each function carefully.
 - Follow the rules — use **ONLY** `if`, `switch`, `while`, `for`, `putchar()`, `getchar()`, `printf()`, `sprintf()`.
 - Do NOT use `goto`, or any standard library functions.
 2. Test your implementation locally using `main.c`.
5. Once all functions are complete — save your changes, then **commit** and **push** to your repository.
6. Go to the **Actions** tab on GitHub and check the test results.

Learning Objectives

After completing this lab, you will:

Use `for` loops to iterate over arrays with known length Use `while` loops to search and converge on solutions Implement array manipulation functions (`min`, `max`, `sum`, `average`) Implement search algorithms (`linear_search`) Implement iterative mathematical algorithms (`heron` - Heron's method) Work with multi-file C projects using header files with include guards.

Implement 6 functions that work with arrays and use iteration:

Function	Purpose	Loop Type	Return Type
<code>find_max()</code>	Find largest element	<code>for</code>	<code>int</code>
<code>find_min()</code>	Find smallest element	<code>for</code>	<code>int</code>
<code>sum_array()</code>	Sum all elements	<code>for</code>	<code>long</code>

Function	Purpose	Loop Type	Return Type
<code>average()</code>	Calculate mean	<code>for</code>	<code>double</code>
<code>linear_search()</code>	Find element index	<code>while</code>	<code>int</code>
<code>heron()</code>	Calculate square root	<code>while</code>	<code>double</code>

Background

For Loops vs While Loops

Use `for` loops when you know the iteration count:

```
for (int i = 0; i < n; i++)
{
    // Process array[i]
}
```

Use `while` loops when you don't know when to stop:

```
int i = 0;
while (i < n)
{
    if (/* found it */)
        return i;
    i++;
}
```

Include Guards

Header files must use include guards to prevent multiple inclusion:

```
#ifndef CODE_H
#define CODE_H

int find_max(int arr[], int n);
// ... more prototypes

#endif
```

Heron's Method for Square Root

Background:

Heron's method iteratively improves an initial guess:

```
next_guess = (current_guess + x / current_guess) / 2.0
```

Stop when consecutive guesses differ by less than epsilon.

Algorithm:

1. Start with initial guess: $\text{guess} = x / 2.0$
2. Use while (1) loop (infinite loop with break):
 - Save current guess: $\text{prev_guess} = \text{guess}$
 - Calculate next guess: $\text{guess} = (\text{guess} + x / \text{guess}) / 2.0$
 - Calculate difference: $\text{diff} = \text{guess} - \text{prev_guess}$
 - Make difference positive manually: if ($\text{diff} < 0$) $\text{diff} = -\text{diff}$;
 - If $\text{diff} < \text{epsilon}$, break the loop
3. Return the final guess

Note: You cannot use fabs() - manually check if negative with if ($x < 0$) $x = -x$;

Example: `heron(4.0, 0.0001);` // Returns 2.0 `heron(9.0, 0.0001);` // Returns 3.0
`heron(2.0, 0.0001);` // Returns 1.414

#Header comment — required in code.c

```
// Your Name  
// CSCI 112 Fall 2026  
// Programming Assignment #1  
// I declare that I am the author of this work, take full responsibility for it, and have d
```

Only for testing

Local Testing (Sandbox)

Create test cases in main.c:

```
#include <stdio.h>  
#include "code.h"  
  
int main()  
{  
    int arr[] = {5, 2, 8, 1, 9};  
    printf("Max: %d\n", find_max(arr, 5));  
    printf("Min: %d\n", find_min(arr, 5));  
    printf("Sum: %ld\n", sum_array(arr, 5));  
  
    float farr[] = {2.0f, 4.0f, 6.0f};  
    printf("Average: %f\n", average(farr, 3));  
}
```

```
printf("Search for 8: %d\n", linear_search(arr, 5, 8));  
printf("Search for 7: %d\n", linear_search(arr, 5, 7));  
  
printf("Square root of 4: %f\n", heron(4.0, 0.0001));  
printf("Square root of 2: %f\n", heron(2.0, 0.0001));  
  
return 0;  
}
```